RAG Commodity Code Classifier

Mohammad Tina

Introduction

Every product that crosses international borders needs a clear description for the receiving country to determine how to tax it. But how do you create a universal language for describing everything from live animals to microchips? You can't list every product in the world.

The Harmonized System (HS), a globally standardized classification system developed by the World Customs Organization. At its core is a 6-digit code structure that covers virtually every tradable product on earth. It organizes all tradable goods into 21 broad sections, covering 99 chapters (numbered 01-99), which then break down into thousands of 6-digit subheadings and, in many countries, further into 8-10-digit national codes.

A simple product like "wheat flour" isn't just one code, it splits into multiple codes based on packaging (wrapped vs. bulk), container size (≤2.5kg vs. larger), and grain type (wheat vs. meslin). The database I am working on alone contains approximately 13,000 national commodity codes, and finding the right one traditionally requires expertise in tariff regulations.

The problem is that users are unable to correctly classify products.

This AI classification system that addresses the challenge from two viewpoints:

1. Decision Support for Customs Brokers

Customs brokers are legal entities authorized to lodge customs declarations in their country and are responsible for accurate product classification. Misclassification leads to financial penalties, shipment delays, and compliance issues. This system acts as an intelligent assistant. When unsure of the exact code, it narrows the search from 13,000 options to 3 high-probability candidates, with full reasoning.

2. Training Data Generation for Predictive Models

The system's structured outputs (classification code, confidence level, reasoning, and full hierarchy path) serve as valuable labelled training data for downstream machine learning models, the information can be used for fraud detection.

RAG Application

The approach combines hybrid retrieval (semantic vector search and keyword matching), legal exclusion rule filtering, and structured LLM prompting to achieve reliable classifications using a local 7B parameter model

The application uses semantic search. with embedding of all 13,000 commodity descriptions using a sentence transformer model, index them in FAISS, and when a user queries "laptop charger," the system would retrieve the most similar descriptions

We tested *"wheat flour"* and one of the results, was chapter 44, which is *"wood flour"*, it should be chapter 11; they looked similar enough to rank highly. But one is a food product (Chapter 11), the other is a wood-processing byproduct(Chapter 44), which is not related to the original prompt, and we don't expect users to write a full description.

Here's how the application retrieval pipeline works:

Step 1: *Semantic Search (FAISS)*
Keep the vector similarity search that was already built.

Step 2*: Keyword Search (BM25)*
Add a traditional keyword ranking algorithm. BM25 scores documents based on term frequency and rarity. If "wheat" appears in the query and "wheat" appears in the commodity description, that's a strong signal regardless of semantic similarity.

Step 3: *Reciprocal Rank Fusion (RRF)*
We use RRF, it looks at positions. If a code ranks #2 in semantic search and #3 in BM25 search, it gets a combined score based on those positions:

RRF_score = 1/(k + semantic_rank) + 1/(k + bm25_rank)

Where k is a constant (typically 60). Codes that appear high in both lists get dramatically boosted. Codes that only appear in one list still show up but with lower priority.

We tested "wheat flour" again:
Semantic search alone:

1. Wood Flour (44050000)
2. Wheat Flour wrapped (11010010)
3. Maize Flour (11022000)

BM25 search:

1. Wheat Flour bulk (11010090)
2. Wheat Flour wrapped (11010010)
3. Wheat gluten (11090000)

After RRF fusion:

1. Wheat Flour wrapped (11010010) ← appeared in both, boosted to top
2. Wheat Flour bulk (11010090) ← appeared in both
3. Wheat gluten (11090000)
4. Wood Flour (44050000) ← pushed way down

By combining both methods, the system caught exact matches while still handling paraphrases.

The Use of LLM

I have always considered an LLM to be a person with expertise in most fields, and with our prompt just makes the person focus on a specific subject.

I set up a local 7B LLM (Qwen2.5 via Ollama) to make the final selection from retrieved candidates. It worked well until I noticed it sometimes returned codes that weren't even in the candidate list, classic hallucination. The fix was two-part: validation that checks if the returned code exists in candidates, and structured prompting that forces the LLM to output specific sections (CLASSIFICATION RESULT, REASONING, HIERARCHY PATH) with strict rules like "*You MUST only select from provided candidates*." .

The system is now in the testing phase, delivering sub-5-second classifications with confidence scores that route uncertain cases to human review. The modular architecture means I can swap models, embeddings, or retrieval strategies through simple config changes as I continue refining accuracy."